

SUPERVISED LEARNING · REGRESSION

수학 없이 이해하는 머신러닝 회귀

공부 시간과 시험 점수처럼 명확한 사례로 배우는
선형회귀, 평가 지표, 다중·다항회귀와 경사하강법

학습 노트 기반 정리본
regression.ipynb · 분류 이전 범위

이 문서의 사용법

공식을 외우기보다 “모델이 무슨 일을 하는가”를 사례와 코드의 순서로 이해하는 문서입니다.

최종 목표

회귀는 과거의 조건과 숫자 결과를 학습해, 새로운 조건의 결과 숫자를 예측하는 방법이라는 흐름을 이해합니다.

목차

01	회귀의 큰 그림: X, y, 학습과 예측
02	학습용·시험용 데이터와 Random Seed
03	선형회귀: 기울기, 절편, 오차
04	평가 지표: MAE, MSE, RMSE, R ²
05	다중선형회귀와 회귀 평면
06	CSV, iloc/loc, X의 데이터 모양
07	다항회귀, 과소적합과 과적합
08	경사하강법과 SGDRegressor
09	잔차와 선형회귀가 잘 맞는 조건
10	데이터 전처리, 검증과 데이터 누수
11	노트북 예제 점검과 실행 주의사항
12	실전 주의점과 전체 복습

범위

로지스틱 회귀는 이름에 회귀가 들어가지만 분류 모델입니다. 이 문서는 노트북에서 로지스틱 회귀가 시작되기 전까지만 다룹니다.

회귀의 큰 그림

회귀는 여러 사례에서 숫자의 변화 흐름을 찾아 새로운 숫자를 예측합니다.

공부 시간 x	실제 시험 점수 y
1시간	20점
2시간	30점
3시간	40점
4시간	50점

모델이 발견할 수 있는 흐름

공부 시간이 1시간 늘어날 때마다 점수가 약 10점 증가한다. 따라서 5시간 공부한 새로운 학생은 약 60점으로 예측한다.

X: 입력 조건

모델에게 알려주는 정보입니다. 공부 시간, 집 크기, 기온처럼 예측에 사용할 조건입니다.

y: 실제 정답

모델이 맞히려는 숫자입니다. 시험 점수, 집값, 매출처럼 최종 예측 대상입니다.

과거 데이터



규칙 학습



새 입력



숫자 예측

회귀인지 판단하는 가장 쉬운 질문

모델의 답이 “얼마인가?”라면 보통 회귀 문제입니다.

- 이 집의 가격은 얼마인가?
- 내일 기온은 몇 도인가?
- 배달은 몇 분 후 도착하는가?
- 시험 점수는 몇 점인가?

한 문장 핵심

회귀는 숫자가 입력되는 문제가 아니라, 최종적으로 숫자의 크기를 예측하는 문제입니다.

학습용과 시험용 데이터

모델이 관계를 이해했는지, 본 데이터를 외웠는지 구분하려면 데이터 역할을 나눠야 합니다.

학습용 train

답을 보며 공부하는 연습문제입니다. 모델이 X와 y의 관계를 찾을 때 사용합니다.

시험용 test

학습 때 보여주지 않은 시험문제입니다. 처음 보는 데이터에도 규칙이 통하는지 확인합니다.

구분	공부 시간	점수	사용 목적
학습용	1, 3, 5시간	20, 38, 59점	관계 학습
시험용	2, 4시간	32, 51점	새 데이터 예측 검사

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

- **test_size=0.2**: 전체의 20%를 시험용으로 사용합니다.
- **random_state=42**: 매번 같은 방식으로 데이터가 섞이도록 고정합니다.

Random Seed

무작위를 없애는 설정이 아닙니다. 같은 무작위 결과를 다시 만들 수 있게 해 모델 변경 전후를 공정하게 비교합니다. 숫자 42 자체에는 특별한 의미가 없습니다.

왜 전부 학습에 쓰면 안 될까?

이미 정답을 본 연습문제를 다시 풀게 하고 실력이 좋다고 판단하는 것과 같습니다. 처음 보는 데이터에서의 성능을 알 수 없습니다.

선형회귀가 학습하는 두 가지

선형회귀는 데이터의 전체 흐름을 나타내는 직선을 찾습니다.

```
model = LinearRegression()
model.fit(X_train, y_train) # 공부
predictions = model.predict(X_test) # 시험
```

기울기

X가 1 증가할 때 예측 y가 얼마나 변하는지 나타냅니다.

0.79

“X가 1 늘면 y가 약 0.79 증가”

절편

X가 0일 때 모델이 예상하는 출발점입니다.

1.57

“X가 0이면 y는 약 1.57”

X=2의 예측

기본값 1.57에서 시작해 0.79를 두 번 더하면 약 3.15가 됩니다. 실제 모델 결과는 반올림 차이를 포함해 약 3.14입니다.

실제값, 예측값, 오차

X	실제값	예측값	빗나간 거리
2	4	3.14	약 0.86
5	5	5.50	0.50

실제 결과에는 수면, 기존 실력, 컨디션처럼 모델이 알지 못하는 조건도 영향을 줍니다. 그러므로 예측과 실제값의 차이는 자연스럽습니다.

한 문장 핵심

선형회귀의 목표는 모든 값을 외우는 것이 아니라, **여러 데이터의 전체적인 오차가 작아지는 직선 규칙**을 찾는 것입니다.

MAE: 평균적으로 얼마나 빗나갔나

MAE는 예측이 정답에서 떨어진 거리를 모두 구한 뒤 평균을 냅니다.

실제값	예측값	빗나간 거리
3	2.5	0.5
-0.5	0	0.5
2	2	0
7	8	1

계산을 말로 읽기

거리 0.5, 0.5, 0, 1을 더하면 2입니다. 데이터 4개로 나누면 **MAE는 0.5**입니다.

왜 방향을 없앨까?

실제보다 10 낮은 예측과 10 높은 예측을 그대로 더하면 서로 지워져 0이 됩니다. MAE는 높고 낮은 방향 대신 **틀린 거리**만 보므로 평균적인 오차 크기를 알 수 있습니다.

시험 점수

MAE가 5라면 평균적으로 약 **5점**씩 빗나갔다는 뜻입니다.

집값

MAE가 1,000만원이라면 평균적으로 약 **1,000만원**씩 차이 난다는 뜻입니다.

한 문장 핵심

MAE는 모델이 정답에서 **평균적으로 얼마나 멀리** 빗나갔는지 원래 단위로 알려줍니다.

MSE와 RMSE: 큰 실수를 더 강하게 보기

MSE는 큰 오차에 더 큰 벌점을 주고, RMSE는 그 결과를 사람이 읽기 쉬운 원래 단위로 되돌립니다.

빗나간 거리	자기 자신과 한 번 더 곱한 값
1	1
2	4
3	9
10	100

MSE

큰 실수를 매우 크게 반영합니다. 하지만 단위도 두 번 곱해져 직접 해석하기 어렵습니다.

RMSE

MSE를 원래 단위로 되돌립니다. 큰 실수를 강조하면서 점, 원, 분처럼 읽을 수 있습니다.

점수와 집값에서 다르게 보이는 값

예측 대상	오차 사례	MSE	RMSE 해석
시험 점수	5, 5, 10, 10점	62.5점 ²	약 7.9점 규모로 빗나감
집값	1,000, 1,000, 2,000, 2,000만원	2,500,000만원 ²	약 1,581만원 규모로 빗나감

자주 하는 오해

MSE가 62.5라고 해서 평균 62.5점을 틀렸다는 뜻이 아닙니다. 사람이 원래 단위로 이해하려면 RMSE를 봅니다.

빠른 비교

MAE: 평균적인 거리 · **MSE**: 큰 실수에 강한 벌점 · **RMSE**: 큰 실수를 강조하면서 원래 단위로 표현

R²: 평균 예측보다 얼마나 나은가

R²는 오차의 단위가 아니라, 모델이 데이터의 흐름을 얼마나 잘 설명하는지 보여줍니다.

비교 기준

실제 점수가 50, 60, 70, 80점이면 평균은 65점입니다. 아무것도 학습하지 않은 방법은 모든 학생에게 65점을 예측할 수 있습니다.

R ²	직관적인 의미
1에 가까움	평균으로 찍는 것보다 데이터 흐름을 매우 잘 설명
0에 가까움	평균값으로 예측하는 것과 비슷
0보다 작음	평균값으로 예측하는 것보다도 못함

학습 점수와 시험 점수를 함께 보기

```
model.score(X_train, y_train) # 학습 데이터 R2
model.score(X_test, y_test)  # 시험 데이터 R2
```

학습 R ²	시험 R ²	의심할 상태
낮음	낮음	관계를 충분히 배우지 못한 과소적합
매우 높음	낮음	학습 데이터를 외운 과적합
높음	높음	새 데이터에도 흐름이 잘 적용될 가능성

노트북 첫 예제의 R²가 음수인 이유

전체 데이터가 5개, 시험 데이터가 2개뿐이라 평가가 매우 불안정합니다. 약 -0.97이라는 결과가 코드 오류를 뜻하는 것은 아닙니다.

조건을 여러 개 사용하기

다중선형회귀는 여러 조건을 동시에 고려해 하나의 결과 숫자를 예측합니다.

공부 시간	과외 시간	시험 점수
2시간	0시간	60점
3시간	1시간	70점
5시간	2시간	85점
8시간	3시간	95점

```
X = np.column_stack((study_hours, tutor_hours))
y = test_scores

model.fit(X, y)
model.predict([[6, 2]]) # 공부 6시간, 과외 2시간
```

노트북 학습 결과

공부 시간 계수 약 4.87, 과외 시간 계수 약 1.94, 절편 약 53.39이며, 공부 6시간·과외 2시간의 예상 점수는 약 **86.5점**입니다.

계수는 이렇게 읽습니다

- 다른 조건이 같다면 공부 시간이 1시간 증가할 때 예측 점수가 약 4.87점 증가
- 다른 조건이 같다면 과외 시간이 1시간 증가할 때 예측 점수가 약 1.94점 증가

선에서 평면으로

조건이 하나면 회귀선, 조건이 두 개면 회귀 평면으로 시각화할 수 있습니다. 파란 점은 실제 학생, 빨간 평면은 모델의 예상이며 점과 평면 사이 거리가 오차입니다.

계수 해석 주의

계수가 크다고 반드시 더 중요한 원인인 것은 아닙니다. 변수마다 단위와 변화 범위가 다르고, 모델이 발견한 관계만으로 인과관계가 증명되지는 않습니다.

조건이 세 개여도 원리는 같다

노트북은 X_1 , X_2 , X_3 세 조건으로 결과 Y 를 만드는 가상 데이터를 사용합니다.

```
np.random.seed(42)
X1 = np.random.normal(0, 1, 100)
X2 = np.random.normal(0, 1, 100)
X3 = np.random.normal(0, 1, 100)

# Y는 X1, X2, X3의 영향과 약간의 무작위 오차를 포함
Y = 2 * X1 + 3 * X2 + 1.5 * X3 + noise
```

데이터를 만든 규칙은 모델에게 직접 알려주지 않습니다. 모델은 100개의 사례를 보고 각 조건의 영향력을 거꾸로 찾아냅니다.

조건	데이터를 만들 때 사용한 영향	모델이 학습한 계수
X_1	약 2	약 1.79
X_2	약 3	약 2.93
X_3	약 1.5	약 1.51

시험 데이터 결과

R^2 는 약 **0.957**, RMSE는 약 **0.72**입니다. 무작위 오차가 섞여 있어 원래 계수와 완전히 같지는 않지만 전체 규칙을 상당히 가깝게 복원했습니다.

그래프로 그릴 수 없다고 학습할 수 없는 것은 아니다

조건 두 개까지는 3차원 평면으로 그릴 수 있지만 조건이 세 개 이상이면 사람이 한 장의 그래프로 보기 어렵습니다. 컴퓨터는 열이 더 많아져도 같은 방식으로 계수를 학습하고 예측할 수 있습니다.

한 문장 핵심

다중선형회귀는 조건의 수가 늘어나도 **다른 조건이 같을 때 각 조건이 결과와 어떻게 함께 변하는지** 학습합니다.

계수 해석에서 놓치기 쉬운 점

계수는 유용하지만 단위, 조건 사이의 중복 정보, 인과관계를 함께 고려해야 합니다.

1. 단위가 다르면 숫자 크기를 직접 비교하기 어렵다

집 크기는 평, 지하철 거리는 미터입니다. 집 크기 계수 1,000과 거리 계수 -2를 숫자만 비교하면 안 됩니다. 10평 차이와 1,000m 차이가 실제 예측에 주는 변화를 함께 봐야 합니다.

2. 비슷한 조건이 함께 있으면 계수가 불안정할 수 있다

집 크기와 방 개수처럼 거의 같은 정보를 가진 두 조건을 동시에 넣으면 모델은 어느 조건에 영향을 얼마나 나눠줄지 혼란스러울 수 있습니다. 데이터가 조금 바뀌었을 때 한 계수는 커지고 다른 계수는 작아질 수 있습니다. 이를 **다중공선성** 문제라고 합니다.

3. 계수는 원인 증명이 아니다

아이스크림 판매량과 물놀이 사고가 함께 증가하더라도 아이스크림이 사고를 일으킨다고 결론 내릴 수 없습니다. 더운 날씨라는 다른 조건이 둘을 함께 증가시킬 수 있습니다.

4. 표준화 후 계수는 비교가 조금 쉬워진다

조건들을 비슷한 숫자 범위로 맞추면 단위 차이의 영향을 줄일 수 있습니다. 그래도 계수가 인과관계나 절대적인 중요도를 증명하는 것은 아닙니다.

실무 해석

계수의 부호와 크기를 읽을 때는 단위, 조건의 실제 변화 범위, 서로 겹치는 정보, 빠진 조건을 함께 확인합니다.

CSV에서 X와 y 꺼내기

회귀 실습에서는 표에서 입력 열과 정답 열을 분리하는 작업이 반복됩니다.

```
dataset = pd.read_csv("../dataset/LinearRegressionData.csv")

X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
```

표현	뜻
iloc	행과 열을 숫자 위치로 선택
:	모든 행
:-1	마지막 열을 제외한 앞의 모든 열
-1	마지막 열

열 이름을 알 때는 다음 코드가 더 읽기 쉽습니다.

```
X = dataset[["hour"]] # 표 형태 유지
y = dataset["score"] # 정답 목록
```

왜 X는 대괄호가 두 겹일까?

X는 조건의 표

```
[[0.5],
 [1.2],
 [1.8]]
```

각 행은 데이터 하나, 각 열은 조건 하나입니다.

y는 정답 목록

```
[10, 8, 14]
```

X의 각 행에 정답 하나가 대응합니다.

새 값 하나도 모델에는 표 형태로 전달합니다.

```
model.predict([[9]]) # 조건 1개인 데이터 1개
model.predict([[6, 2]]) # 조건 2개인 데이터 1개
```

한 문장 핵심

X는 여러 데이터와 여러 조건을 담은 표이고, y는 각 데이터에 대응하는 정답 목록입니다.

직선으로 부족할 때 곡선 사용하기

공부 시간이 늘어도 점수 상승 폭이 항상 같지는 않습니다. 처음에는 빠르게 오르고 나중에는 완만해질 수 있습니다.

공부 시간	점수	변화의 느낌
1시간	20점	출발
3시간	55점	빠르게 상승
5시간	84점	상승 폭 감소
7시간	94점	거의 완만

```
polyreg = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
polyreg.fit(X, y)
```

차수 degree	표현 능력
1	직선
2	완만하게 휘는 곡선
3 이상	더 복잡하게 휘는 곡선

노트북 결과

2차 다항회귀는 공부 3.8시간의 성적을 약 **50.7점**으로 예측하고, 전체 학습 데이터의 R^2 는 약 **0.976**입니다.

과소적합

모델이 너무 단순하여 학습 데이터와 시험 데이터 모두에서 흐름을 잘 잡지 못합니다.

과적합

곡선을 지나치게 복잡하게 만들어 학습 데이터의 우연한 흔들림까지 외웁니다.

좋은 모델

학습 데이터를 완벽히 통과하는 모델이 아니라, 처음 보는 시험 데이터에서도 전체 흐름을 잘 적용하는 모델입니다.

다항회귀 점수는 어떻게 검증할까?

노트북의 R^2 0.976은 모델을 학습시킨 바로 그 데이터에서 계산한 점수입니다.

```
polyreg.fit(X, y)
r_squared = polyreg.score(X, y) # 학습에 사용한 데이터로 평가
```

이미 답을 보며 곡선을 만든 뒤 같은 문제로 시험을 치른 결과이므로, 처음 보는 데이터에서도 0.976이 나온다고 보장할 수 없습니다.

더 공정한 평가

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

polyreg.fit(X_train, y_train)
print(polyreg.score(X_train, y_train))
print(polyreg.score(X_test, y_test))
```

결과	가능한 해석
학습·시험 점수 모두 낮음	곡선이 관계를 충분히 표현하지 못하는 과소적합 가능성
학습 점수만 매우 높음	학습 데이터의 흔들림까지 외운 과적합 가능성
둘 다 높고 차이가 작음	새 데이터에도 흐름이 적용될 가능성

차수는 시험 데이터로 계속 고르면 안 된다

degree 1, 2, 3을 시험 데이터에 반복 적용해 가장 좋은 것을 고르면 시험문제를 보며 모델을 선택하는 셈입니다. 차수 선택에는 학습 데이터 내부의 검증 또는 교차검증을 사용하고, 시험 데이터는 마지막 평가에 남겨둡니다.

한 문장 핵심

모델의 복잡도를 선택하는 점수와 최종 성능을 보고하는 시험 점수는 역할을 분리해야 합니다.

오차를 줄이는 방향으로 반복 수정

경사하강법은 적절한 기울기와 절편을 한 번에 맞히지 않고, 예측과 수정을 반복해 찾아갑니다.

예측

→

오차 확인

→

조금 수정

→

다시 예측

기울기 탐색 예시

실제 흐름이 "1시간마다 약 10점 증가"라면 모델은 $2 \rightarrow 5 \rightarrow 8 \rightarrow 9.5 \rightarrow 9.9$ 처럼 오차가 작아지는 방향으로 값을 수정할 수 있습니다.

산에서 낮은 곳 찾기

현재 위치에서 아래로 내려가는 방향을 확인하고 조금 움직인 뒤, 새로운 위치에서 다시 확인하는 과정을 반복한다고 생각하면 됩니다. 가장 낮은 곳은 예측 오차가 가장 작은 상태입니다.

```
sr = SGDRegressor(
    max_iter=200,
    eta0=1e-4,
    random_state=0
)
sr.fit(X_train, y_train)
```

설정	의미	주의
eta0	한 번에 수정하는 크기, 즉 학습률	너무 크면 지나치고, 너무 작으면 오래 걸림
max_iter	최대 반복 횟수	부족하면 안정적인 값에 도착하기 전에 종료
random_state	무작위 과정 재현	비교 실험에서 같은 값 사용

노트북의 수렴 경고

"Maximum number of iteration reached before convergence"는 200번 수정했지만 아직 충분히 안정적인 지점에 도착하지 못했다는 뜻입니다.

잔차: 오차를 방향까지 살펴보기

MAE와 RMSE는 오차를 하나의 숫자로 요약하지만, 잔차는 각 예측이 어느 방향으로 얼마나 틀렸는지 보여줍니다.

실제 점수	예측 점수	잔차	해석
80	70	+10	모델이 10점 낮게 예측
60	65	-5	모델이 5점 높게 예측
90	90	0	정확히 예측

좋은 잔차 그림

예측값이 작거나 크더라도 잔차가 0 위아래에 특별한 모양 없이 흩어져 있으면 모델이 놓친 뚜렷한 패턴이 적다는 신호입니다.

문제가 보이는 모양

- **활처럼 휨:** 실제 관계가 곡선인데 직선 모델을 사용했을 수 있습니다.
- **갈때기처럼 퍼짐:** 값이 커질수록 오차가 커져 예측의 안정성이 달라질 수 있습니다.
- **한쪽에 치우침:** 모델이 전반적으로 높게 또는 낮게 예측할 수 있습니다.
- **멀리 떨어진 점:** 이상치나 데이터 오류를 확인해야 합니다.

```
residuals = y_test - y_pred
plt.scatter(y_pred, residuals)
plt.axhline(0, color="red", linestyle="--")
```

한 문장 핵심

평가 점수 하나만 보지 말고 잔차에 남은 모양을 확인하면 모델이 놓친 관계를 발견할 수 있습니다.

선형회귀가 잘 맞으려면 확인할 것

아래 항목은 복잡한 공식을 외우기 위한 것이 아니라 결과를 지나치게 믿지 않기 위한 점검표입니다.

확인사항	쉬운 의미	확인 방법
관계의 모양	x가 변할 때 y의 평균 흐름이 대체로 직선인가?	산점도와 잔차 그림
오차의 독립성	한 예측의 오차가 다음 예측에 연달아 영향을 주지 않는가?	시간 순서 데이터인지 확인
오차 크기의 안정성	작은 값과 큰 값에서 오차의 퍼짐이 지나치게 달라지지 않는가?	잔차 그림의 깔때기 모양 확인
이상치	소수의 특이한 값이 회귀선을 끌어당기지 않는가?	산점도, 잔차, 원본 데이터 확인
조건의 중복	서로 거의 같은 정보를 가진 x들이 계수를 불안정하게 만들지 않는가?	특성 간 관계와 계수 변화 확인

예측과 통계 해석의 차이

오차가 완벽한 모양을 이루지 않아도 예측 모델로 사용할 수는 있습니다. 다만 계수의 통계적 의미나 불확실성을 엄밀하게 해석하려면 위 조건을 더 주의 깊게 확인해야 합니다.

시간 데이터

매출, 주가, 날씨처럼 시간 순서가 있는 데이터는 무작위 분할이 미래 정보를 과거 학습에 섞을 수 있습니다. 과거로 학습하고 더 나중 시점으로 시험하는 방식이 필요합니다.

결측치와 범주형 조건 처리

실제 데이터는 모든 값이 숫자로 완벽하게 채워져 있지 않습니다.

결측치

면적	층수	역과 거리	집값
24평	10층	500m	5억원
32평	값 없음	800m	6억원

값이 없으면 해당 행을 제거하거나, 학습 데이터에서 계산한 중앙값·평균 등으로 채울 수 있습니다. 시험 데이터의 값까지 함께 보고 채우는 기준을 만들면 데이터 누수가 됩니다.

범주형 조건

지역, 건물 유형, 요일처럼 글자로 된 조건은 모델이 사용할 숫자 열로 바뀌야 합니다.

지역	강남 열	마포 열	송파 열
강남	1	0	0
마포	0	1	0
송파	0	0	1

지역을 강남=1, 마포=2, 송파=3으로만 바꾸면 모델이 송파가 강남보다 세 배 크다는 잘못된 숫자 관계를 학습할 수 있습니다. 그래서 각 종류를 별도의 0/1 열로 만드는 방식이 흔히 사용됩니다.

Pipeline으로 순서 묶기

결측치 처리, 표준화, 다항 특성 생성과 모델 학습을 Pipeline으로 묶으면 학습 데이터에서만 기준을 만들고 시험 데이터에 같은 처리를 적용하기 쉬워집니다.

한 문장 핵심

전처리도 모델 학습의 일부이므로 기준은 학습 데이터에서만 만들고 시험·새 데이터에는 그대로 적용합니다.

검증 데이터와 교차검증

모델 종류나 다항 차수, 학습률을 선택하면서 시험 데이터를 반복해서 보면 시험 데이터까지 외우게 됩니다.

학습 train

계수와 규칙을 학습

검증 validation

모델과 설정을 선택

시험 test

선택이 끝난 뒤 최종 평가

데이터가 적을 때 교차검증

학습 데이터를 여러 묶음으로 나누고, 번갈아 검증 역할을 맡깁니다. 한 번의 우연한 분할보다 여러 분할에서 안정적인 모델을 선택할 수 있습니다.

회차	학습 묶음	검증 묶음
1	A, B	C
2	A, C	B
3	B, C	A

데이터 누수의 대표 사례

- 전체 데이터로 표준화 기준을 만든 뒤 학습·시험을 분리
- 시험 데이터의 결측치까지 함께 보고 평균을 계산
- 시험 R^2 가 가장 높아지도록 degree를 반복 선택
- 미래 매출을 예측하면서 미래 정보가 포함된 열을 X로 사용

시험 데이터의 역할

시험 데이터는 학습 방법을 고치는 힌트가 아니라, 모든 선택이 끝난 모델이 처음 보는 상황에서 얼마나 작동하는지 확인하는 최종 시험입니다.

결과를 믿기 전에 확인할 것

1. 학습 범위 밖의 예측

1~10시간의 공부 데이터로 6시간을 예측하는 것은 비교적 자연스럽습니다. 하지만 100시간을 넣으면 직선을 그대로 연장해 1,000점 같은 비현실적인 결과가 나올 수 있습니다.

범위 안: 보간

이미 본 범위의 중간을 예측합니다. 주변 데이터가 있어 상대적으로 안정적입니다.

범위 밖: 외삽

본 적 없는 범위까지 규칙을 연장합니다. 현실의 제한을 몰라 위험할 수 있습니다.

2. 이상치

1~4시간의 점수가 20, 30, 40, 50점인데 5시간 학생만 5점이라면 마지막 값은 일반 흐름에서 크게 벗어납니다. 입력 실수, 특별한 사건, 실제 회귀 사례 중 무엇인지 먼저 확인해야 합니다.

3. 실제값·예측값 그래프

- 점이 대각선 가까이에 있을수록 실제값과 예측값이 비슷합니다.
- 대각선 위쪽이면 실제보다 높게, 아래쪽이면 낮게 예측했습니다.
- 선에서 멀리 떨어진 점은 크게 틀린 사례입니다.

4. 계수와 단위

집 크기는 평, 지하철 거리는 미터처럼 단위가 다릅니다. 계수 숫자만 비교해 "가장 중요한 특성"을 결정하면 잘못 해석할 수 있습니다. 필요하면 조건들의 기준을 맞추는 표준화를 사용합니다.

인과관계 주의

회귀가 "함께 변하는 관계"를 찾았다고 해서 한 조건이 결과를 직접 일으켰다는 사실까지 증명하는 것은 아닙니다.

조건이 많을 때 과적합 줄이기

조건이 매우 많거나 서로 비슷하면 계수가 지나치게 커지고 새 데이터에서 불안정해질 수 있습니다.

집값 예측에 면적, 방 수, 거실 크기, 창문 수, 건물 연식 등 많은 조건을 넣었더니 학습 데이터는 거의 완벽하지만 시험 데이터에서는 크게 틀릴 수 있습니다.

규제는 모델이 오차만 줄이려고 지나치게 큰 계수를 만드는 것을 억제해 더 단순하고 안정적인 규칙을 찾도록 돕습니다.

Ridge 회귀

계수들을 전반적으로 작게 눌러 여러 조건을 유지하면서 과도한 영향력을 줄입니다.

Lasso 회귀

일부 계수를 0까지 줄일 수 있어 사용하지 않는 조건을 선택하는 효과가 생길 수 있습니다.

```
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0)
lasso = Lasso(alpha=0.1)

ridge.fit(X_train, y_train)
lasso.fit(X_train, y_train)
```

alpha가 커질수록 계수를 더 강하게 제한합니다. 너무 약하면 과적합을 줄이지 못하고, 너무 강하면 필요한 관계까지 약하게 만들어 과소적합될 수 있습니다. 적절한 값은 교차검증으로 선택합니다.

표준화

Ridge와 Lasso는 조건의 숫자 단위에 영향을 받으므로 보통 표준화와 함께 사용합니다.

범위 표시

Ridge와 Lasso는 원본 노트북에는 없지만, 선형회귀의 과적합과 계수 불안정을 이해하기 위한 필수 보강 개념입니다.

당뇨병 예제는 회귀가 아니라 분류

노트북은 diabetes.csv의 Outcome을 LinearRegression으로 예측하지만 Outcome은 0 또는 1의 클래스입니다.

값	의미	데이터 성격
0	당뇨병 음성	클래스 레이블
1	당뇨병 양성	클래스 레이블

LinearRegression을 사용하면 생기는 문제

- 예측값이 -0.1이나 1.2처럼 클래스 범위를 벗어날 수 있습니다.
- R^2 , MSE, 실제값-예측값 산점도는 이진 분류의 핵심 평가 방식을 보여주지 못합니다.
- 양성 환자를 놓친 경우와 정상인을 양성으로 잘못 경고한 경우를 구분하지 못합니다.

더 적절한 흐름

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix, classification_report
)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

정확도만 보지 말고 혼동 행렬, 양성 클래스의 정밀도·재현율·F1을 확인해야 합니다. 의료 데이터에서는 실제 환자를 놓치는 실수의 중요성도 고려해야 합니다.

특성 중요도 해석

원본 예제는 표준화하지 않은 선형회귀 계수의 절댓값을 중요도로 사용합니다. 혈당, BMI, 나이는 단위가 다르므로 계수 크기만으로 건강 지표 중요도를 비교하면 왜곡될 수 있습니다.

현재 회귀 노트북 실행 전 확인사항

항목	현재 상태	권장 조치
데이터 경로	dataset/..., ../dataset/..., ../dataset/... 혼용	os.getcwd() 확인 후 한 기준으로 통일
당뇨 Outcome	0/1 대상에 LinearRegression 사용	분류 모델과 분류 평가 지표 사용
다항회귀 평가	학습한 전체 데이터에서 R ² 계산	학습·시험 분리 또는 교차검증
SGDRegressor	200회 이전에 수렴하지 않았다는 경고 저장	표준화, 학습률, 반복 횟수 점검
셀 상태	X, y, model 이름을 여러 예제에서 반복 사용	새 커널에서 위에서부터 실행하고 예제별 변수명 사용
첫 단순회귀	시험 데이터가 2개뿐	R ² 를 일반 성능으로 해석하지 않기

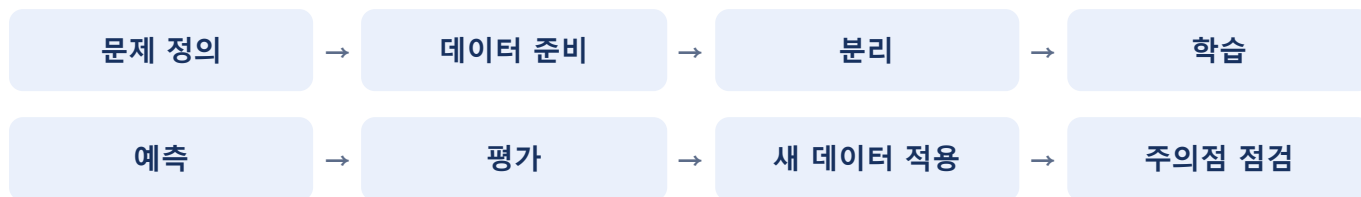
경로가 실행 위치에 따라 달라지는 이유

- 작업 폴더가 C:\work이면 dataset/file.csv가 맞습니다.
- 작업 폴더가 C:\work\supervisedLearning이면 ../dataset/file.csv가 맞습니다.
- 현재 프로젝트에는 회귀용 CSV 세 개가 C:\work\dataset에 존재합니다.

셀 실행 순서

Jupyter는 이전 셀의 변수를 기억합니다. 중간 셀만 실행하면 화면상 가까운 코드가 아니라 오래전에 만든 X_train이나 model을 사용할 수 있습니다. 커널을 재시작하고 위에서부터 실행하는 검사가 중요합니다.

회귀 전체 흐름 한 장 복습



1. **y 정하기**: 집값, 점수, 매출처럼 예측할 숫자를 정합니다.
2. **x 정하기**: 면적, 공부 시간, 방문객 수처럼 예측에 사용할 조건을 정합니다.
3. **데이터 분리**: 학습용으로 관계를 배우고 시험용으로 새 데이터 성능을 검사합니다.
4. **모델 학습**: `fit()`으로 입력 조건과 결과 사이의 흐름을 찾습니다.
5. **예측**: `predict()`로 시험 데이터와 새로운 데이터의 숫자를 예상합니다.
6. **평가**: MAE·RMSE로 오차 크기, R^2 로 평균 예측 대비 흐름 설명력을 봅니다.
7. **검토**: 과적합, 이상치, 단위, 학습 범위 밖 입력을 확인합니다.

최종 체크리스트

- ✓ x와 y의 역할을 설명할 수 있다.
- ✓ 학습용과 시험용 데이터를 나누는 이유를 설명할 수 있다.
- ✓ 기울기, 절편, 오차를 일상적인 말로 설명할 수 있다.
- ✓ MAE, MSE, RMSE, R^2 의 차이를 구분할 수 있다.
- ✓ 단순·다중·다항회귀의 차이를 예시로 설명할 수 있다.
- ✓ 경사하강법을 “예측과 수정의 반복”으로 설명할 수 있다.
- ✓ 과적합, 이상치, 외삽이 왜 위험한지 설명할 수 있다.
- ✓ 잔차 그림에서 곡선·깎때기·이상치를 찾을 수 있다.
- ✓ 전처리 기준을 학습 데이터에서만 만들어야 하는 이유를 안다.
- ✓ 검증 데이터와 시험 데이터의 역할을 구분할 수 있다.
- ✓ 0/1 레이블 대상은 일반적으로 회귀가 아니라 분류 문제임을 안다.

최종 한 문장

회귀는 과거의 입력 조건 **x**와 실제 숫자 **y**를 학습하여, 새로운 조건의 결과 숫자를 예측하는 지도학습 방법입니다.

노트북 코드 빠른 참조

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. 데이터 불러오기
dataset = pd.read_csv("../dataset/LinearRegressionData.csv")

# 2. 입력과 정답 분리
X = dataset[["hour"]]
y = dataset["score"]

# 3. 학습용과 시험용 분리
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

# 4. 모델 생성과 학습
model = LinearRegression()
model.fit(X_train, y_train)

# 5. 시험 데이터 예측
y_pred = model.predict(X_test)

# 6. 평가
print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", mean_squared_error(y_test, y_pred) ** 0.5)
print("R²:", r2_score(y_test, y_pred))

# 7. 새로운 데이터 예측
print("9시간 예상 점수:", model.predict([[9]])[0])
```

코드를 읽는 순서

read_csv로 가져오기 → X와 y 나누기 → train_test_split으로 역할 나누기 → fit으로 공부하기 → predict로 답하기 → 지표로 검사하기

노트북에서 기억할 실제 결과

예제	결과
공부 시간 전체 데이터 선형회귀	9시간 예상 약 93.8점, R ² 약 0.953
공부·과외 시간 다중회귀	6시간·2시간 예상 약 86.5점, 학습 R ² 약 0.961
X1·X2·X3 가상 다중회귀	시험 R ² 약 0.957, RMSE 약 0.72
2차 다항회귀	3.8시간 예상 약 50.7점, 학습 데이터 R ² 약 0.976

끝 · 다음 학습 범위는 분류이며, 로지스틱 회귀부터 시작합니다.